NumPy



Numpy

- NumPy provides an N-dimensional array type, ***ndarray***, that is a (usually fixed-size) multidimensional container of items of the same type
  - Python object for numerical operations
  - Array oriented computing
  - Extension package to Python for multi-dimensional arrays
  - Memory efficient container that provides fast numerical operations
  - Designed for scientific computation

    >>conda install numpy

    Most applied convention

```
import numpy as np
a=np.array([0, 1, 2, 3])
a
```

```
array([0, 1, 2, 3])
```

  - Numpy Reference Documentation
    - np.array?

```
Docstring:
array(object, dtype=None, copy=True, order='K', subok=False, ndmin=0)

Create an array.

Parameters
```

Arrays

[ndarray]

Numpy

- Creating the array
  - One dimension (1D)
    - **array()** method

```
1 import numpy as np
2 a=np.array([0, 1, 2, 3])
3 a
```

```
array([0, 1, 2, 3])
```

    - explicit creation of a numpy array
      - An **ndarray** (N-dimensional array) is a fixed-size multidimensional container of items of the same type and size.
      - This object has several methods and attributes.

Numpy

- Two useful properties of a ndarray:
  - ndim
    - provides the number of array dimensions

```
1 a.ndim
```

```
1
```

  - shape
    - provides the number of elements for each dimension in a tuple form
      - A one-element tuple has a comma at the end

```
a.shape
```

```
(4,)
```

It means that the array has four elements in a single dimension

  - len()
    - Python function
    - Returns the number of items in a container

```
len(a)
```

```
4
```


Numpy

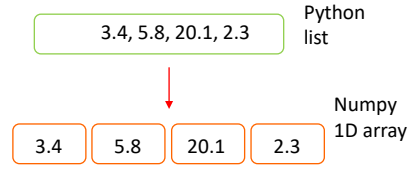
Numpy Array	Python List
Numpy library is required to create a numpy array. **import numpy as np**	Python's core, no additional library is required. It is a built-in function of Python.
It contains similar datatypes	It may contain different data types
Element wise operation	No element wise operation
The creation of N-dimensional array is straightforward with numpy.array()	By default the array is 1 dimensional. The process to create N-dimensions array is not direct.
Requires less memory when compared with a Python list.	Requires more memory than a Numpy array
Items are stored in memory in a sequential manner.	Items are stored in memory at a random location (less efficient)
Numpy array computes faster than a Python List.	
It allows some additional mechanisms (e.g. broadcast)	



Exercise 7

Exercise 7

- Direct conversion of a list of numeric values into a 1D-NumPy array.
  - [3.4, 5.8, 20.1, 2.3]





End - Exercise 7

Numpy

- Multidimensional Arrays
  - 2D (array of 1D arrays)
    - **Important !!**
      - Represent tabular data
    - The array is **defined by row**

```
b=np.array([[0,1,2],[3,4,5]])
b
```

```
array([[0, 1, 2],
       [3, 4, 5]])
```

    - ndim
      - Get dimensions

```
b.ndim
```

```
2
```

    - shape
      - Get a tuple with the numbers of rows/columns

```
1 b.shape
```

```
(2, 3)
```

    - len()
      - Returns, only, the size of the first dimension
      - May be useful in specific situations (e.g. number of instances)

```
1 len(b)
```

```
2
```

Numpy

- Multidimensional Arrays
  - 3D (array of 2D arrays)

```
c=np.array([[[1,4,9],[2,5,4]],[[3,4,1],[4,7,2]],[[3,4,1],[4,7,2]],[[3,4,1],[4,7,2]]])
c
```

```
array([[[1, 4, 9],
        [2, 5, 4]],

       [[3, 4, 1],
        [4, 7, 2]],

       [[3, 4, 1],
        [4, 7, 2]],

       [[3, 4, 1],
        [4, 7, 2]]])
```

  - ndim

```
c.ndim
```

```
3
```

  - shape
    - number of elements for each dimension in a tuple form

```
c.shape
```

```
(4, 2, 3)
```

  - len()
    - the length of the first dimension

```
len(c)
```

```
4
```


Numpy – Arrays Creation

- Creating arrays
  - Actually, in practice the creation of arrays is **rarely supported** in the direct introduction of values
  - Some common methods:
    - np.arange()
      - uniform spacing between elements
      - starts in 0 and ends in n-1 (default)

```
a=np.arange(10)
a
```

```
array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
```

    - np.arange(start,end,step)

```
a=np.arange(1,15,2)     # start, end, step
a
```

```
array([ 1,  3,  5,  7,  9, 11, 13])
```

Numpy – Arrays Creation

- Creating arrays
  - *np.linspace(start, end, points)*
    - allows to define the number of points between a starting and end point

```
a=np.linspace(0,1,6)
a
```

```
array([0. , 0.2, 0.4, 0.6, 0.8, 1. ])
```

  - *np.ones()*
    - the argument is a tuple (nrows, ncols)

```
a=np.ones((3,4))     # (3,4) is a tupple
a
```

```
array([[1., 1., 1., 1.],
       [1., 1., 1., 1.],
       [1., 1., 1., 1.]])
```

  - *np.zeros()*
    - the argument is a tuple (nrows, ncols)

```
a=np.zeros((3,4))     # (3,4) is a tupple
a
```

```
array([[0., 0., 0., 0.],
       [0., 0., 0., 0.],
       [0., 0., 0., 0.]])
```


Numpy – Arrays Creation

- Creating arrays
  - *np.eye(n_dimensions)*
    - identity matrix
      - important to some algebra operations

```
a=np.eye(4)
a
```

```
array([[1., 0., 0., 0.],
       [0., 1., 0., 0.],
       [0., 0., 1., 0.],
       [0., 0., 0., 1.]])
```

  - *np.diag(elements)*
    - A diagonal matrix is a matrix in which all the entries outside the main diagonal are zero

```
1 import numpy as np
2 a=np.diag((1,2,3,4))
3 a
```

```
array([[1, 0, 0, 0],
       [0, 2, 0, 0],
       [0, 0, 3, 0],
       [0, 0, 0, 4]])
```

```
1 a.shape
```

```
(4, 4)
```

Numpy – Arrays Creation

- Creating arrays
  - *reshape()*
    - This method is often applied with arrange() in order to create a 2D array

```
1 import numpy as np
2 a=np.arange(80)
3 print(a)
```

```
[ 0  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20 21 22 23
 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47
 48 49 50 51 52 53 54 55 56 57 58 59 60 61 62 63 64 65 66 67 68 69 70 71
 72 73 74 75 76 77 78 79]
```

    - 2D array based on reshape():

```
1 a.reshape(8,10)
```

```
array([[ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9],
       [10, 11, 12, 13, 14, 15, 16, 17, 18, 19],
       [20, 21, 22, 23, 24, 25, 26, 27, 28, 29],
       [30, 31, 32, 33, 34, 35, 36, 37, 38, 39],
       [40, 41, 42, 43, 44, 45, 46, 47, 48, 49],
       [50, 51, 52, 53, 54, 55, 56, 57, 58, 59],
       [60, 61, 62, 63, 64, 65, 66, 67, 68, 69],
       [70, 71, 72, 73, 74, 75, 76, 77, 78, 79]])
```




Exercise 8

Exercise 8

- Create an array with 3 elements such that:
  - 1st element: a matrix (3x3) of ones
  - 2nd element: a matrix (3x3) of zeros
  - 3rd element: a matrix (3x3) where all elements are zero except the main diagonal (1,2,3)

Exercise 8

- Get the dimensions of this array as well as the number of elements in each dimension

```
array([[[1., 1., 1.],
        [1., 1., 1.],
        [1., 1., 1.]],

       [[0., 0., 0.],
        [0., 0., 0.],
        [0., 0., 0.]],

       [[1., 0., 0.],
        [0., 2., 0.],
        [0., 0., 3.]]])
```



End - Exercise 8

Numpy – Arrays Creation

- Random arrays
  - ***np.random.default_rng()***
    - Several methods to generate random sampling
    - Uniform Distribution in the interval [0,1[

```
import numpy as np
rng=np.random.default_rng()
```

```
x=rng.random(4)
x
```

```
array([0.4056831 , 0.20430855, 0.48246344, 0.68566462])
```

    - random integers in a specific interval

```
y=rng.integers(0,20,size=4)
y
```

```
array([ 0,  5, 15,  3])
```

Numpy – Arrays Creation

- Random arrays
  - normal distribution with specific values for mean and standard deviation

```
z=rng.normal(0,1,size=10)
z
```

```
array([-1.88831307, -0.77684015,  0.08144755,  1.462475  ,  0.46226486,
       -0.52582579, -2.31487399,  1.48953866, -1.3035807 ,  0.21115892])
```

  - shuffle de uma lista

```
lista=[1,2,3,4,5]
rng.shuffle(lista)
lista
```

```
[4, 1, 3, 2, 5]
```


Numpy – Arrays Creation

- Random arrays
  - select random elements from a list

```
amostra=rng.choice([10,5,6,7,8,9],size=3, replace=False)
amostra
```

```
array([ 5,  7, 10])
```



Exercise 9

Exercise 9

- Explore functions to create arrays, namely generate:
  - an array with all integers from 0 to 6
  - the same array with *linspace()*

Exercise 9

- an identity matrix with dimension 4

- an array with 5 random elements generated based on the Gaussian Distribution

Exercise 9

- Create a 5x5 array with random values in the interval [0,1[, rounded to 2 decimals, and find the respective minimum and maximum.
  - Tip: round(); max(); min()



End - Exercise 9

Basic Data Types

Numpy – Basic Data Types

- Numpy auto-detects the data type from input
  - Frequently NumPy works with float numbers
  - However, it is possible to specify the data type (dtype)

```
a=np.array([1,2,3])
print(a)
print(a.dtype)

a=np.array([1,2,3],dtype=float)
print(a)
print(a.dtype)
```

```
[1 2 3]
int32
[1. 2. 3.]
float64
```

  - Numpy allows different data types: https://www.tutorialspoint.com/numpy/numpy_data_types.htm

Indexing/Slicing

Numpy - Indexing

- NumPy arrays allow indexing

```
a=np.arange(10)
a
```

```
array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
```

  - start in 0
  - negative index starts counting from the end of the array

```
print(a[0])     # index starts in 0
print(a[2])
print(a[-1])    # negative index starts from the end
```

```
0
2
9
```


Numpy - Indexing

- Indexing can be performed based on an array of integers
  - this possibility allows a direct modification of some of the original values

```
1 import numpy as np
2 a=np.arange(10)
3 a[[2,4]]=[50,50]
4 print(a)
```

```
[ 0  1 50  3 50  5  6  7  8  9]
```

- The syntax [: : -1] is very frequently applied to easily reverse the order of an array

```
a=np.arange(6)
print(a)
a[::-1]
```

```
[0 1 2 3 4 5]

array([5, 4, 3, 2, 1, 0])
```

  - numpy arrays allow a much more powerful manipulation than python lists

Numpy - Indexing

- Multidimensional arrays
  - Indexing is based on one index per dimension
    - In 2D arrays the first dimension refers to rows and the second dimension to columns

```
1  a=np.diag((1,2,3))
2
3  print(a[0,0])
4  print(a[1,1])
```

```
1
2
```

```
array([[1, 0, 0],
       [0, 2, 0],
       [0, 0, 3]])
```


Numpy - Indexing

- Multidimensional arrays
  - In some specific situations is possible to isolate values without specifying the indices for all dimensions
    - This approach lets to slice a multidimensional array as the elements of the unspecified dimensions can be easily retrieved

```
array([[1, 0, 0],
       [0, 2, 0],
       [0, 0, 3]])
```

```
1  a[1]
```

```
array([0, 2, 0])
```

In this case all the elements of the second row (index 1)

- Useful methods:
  - np.argsort()
    - provides the indices of the sorted original array elements (default: ascending order)
    - np.argsort() ≠ np.sort()



Exercise 10

Exercise 10

- The script should get the indices of the sorted elements of a given array:
  - [5 , 3 , 9 , 1]
  - Print the sorted array:



End - Exercise 10

Numpy - Slicing

- Arrays can be sliced
  - from the start to end position
    - [start:end:step]
  - Element of the last position **is not included**

```
1 a=np.arange(9)
2 print('original array:  ', a)
```

```
original array:   [0 1 2 3 4 5 6 7 8]
```

```
1 print('sliced array - step 1:  ', a[4:7:1])
```

```
sliced array - step 1:   [4 5 6]
```

```
1 print('sliced array - step 2:  ', a[4:7:2])
```

```
sliced array - step 2:   [4 6]
```

  - from the start to a specific position

```
1 print('sliced array - first four:  ', a[:4])
```

```
sliced array - first four:   [0 1 2 3]
```

    - the last element (last index) **is not included**

Numpy - Slicing

- from a given position (included) to the end

```
1 print('sliced array - last four', a[5:])
```

```
sliced array - last four [5 6 7 8]
```

- The three slice components are not mandatory, when they are not defined the following values are assumed by default:
  - **Start** is 0
  - **End** is the last element
  - **Step** is 1

Numpy – Indexing/ Slicing

- Often in practice slicing is applied:

```
1 a=np.arange(9)
2 print('original array:  ', a)
```

```
original array:   [0 1 2 3 4 5 6 7 8]
```

- to get some part of the array
- to change some values in the array

```
1 a[4:]=10
2 print('Modified array:  ', a)
```

```
Modified array:   [ 0  1  2  3 10 10 10 10 10]
```

- This is a very straightforward notation for working with Numpy Arrays

Numpy – Copy/View

- Slicing: Copies/Views
  - a slicing operations creates a **view** of the original data which allows to save memory and time but can also originate some problems.

```
1 a=np.arange(10)
2 print('original array  :', a)
```

```
original array  : [0 1 2 3 4 5 6 7 8 9]
```

```
1 b=a[::2]
2 print('derived array  :  ',b )
```

```
derived array  :   [0 2 4 6 8]
```

```
1 print('is b a view of a?  : ',np.may_share_memory(a,b))
```

```
is b a view of a?  :  True
```

  - which implies that:

```
1 b[0]=100
2 print('original array :  ',a)
```

```
original array :  [100   1   2   3   4   5   6   7   8   9]
```

```
1 print('derived array :  ',b)
```

```
derived array :  [100   2   4   6   8]
```

b is a view of **a**, actually all changes in **b (a)** affect **a (b)**

Numpy – Copy/View

- Slicing: Copies/Views
  - Create a copy
    - np.copy()

```
1 c=np.copy(a[::2])
2 print('copied array :  ',c)
```

```
copied array :  [100   2   4   6   8]
```

  - which assures independent arrays:

```
1 print('Is c a view of a?  : ',np.may_share_memory(a,c))
```

```
Is c a view of a?  :  False
```

```
1 c[0]=800
2 print(c)
```

```
[800   2   4   6   8]
```

```
1 print(a)
```

```
[100   1   2   3   4   5   6   7   8   9]
```

c and **a** are independent !



Exercise 11

Exercise 11

- create a bidimensional array (6 x6) from 0 to 35
  - tip: combine arange() with reshape ((n_lines,n_columns))

```
[[ 0  1  2  3  4  5]
 [ 6  7  8  9 10 11]
 [12 13 14 15 16 17]
 [18 19 20 21 22 23]
 [24 25 26 27 28 29]
 [30 31 32 33 34 35]]
```

```
a=np.arange(36).reshape(6,6)
print(a)
```

  - based on slicing isolate:

green elements blue elements orange elements



End - Exercise 11

Numerical Operations on Arrays



Numpy – Numerical Operations

- **Elementwise** operations
  - operation on each element of an array
  - all arithmetic operations are elementwise
    - multiplication (scalar)

```
a=np.array([1,2,3,4])
a=a*2
print(a)
```

```
[2 4 6 8]
```

    - Sum/Difference

```
1 a=np.array([1,2,3,4])
2 b=np.arange(4)
3 print('Sum Elementwise:', a+b)
4 print('Dif Elementwise:', a-b)
```

```
Sum Elementwise: [1 3 5 7]
Dif Elementwise: [1 1 1 1]
```

    - Multiplication
      - Important: * is not a matrix multiplication (it is an elementwise operation)
      - Matrix multiplication **.dot()**

```
a=np.array([1,2,3,4])
print(a)

c=np.array([2,2,3,3])
a=a*c
print(a)
```

```
[1 2 3 4]
[ 2  4  9 12]
```


Numpy – Numerical Operations

- **np.dot()** is applied to compute inner products of arrays, to multiply an array by a matrix, and to multiply matrices.
  - Multiply 2 matrices:

```
import numpy as np

x=np.array([[1,2,3],[2,3,1]])
y=np.array([[4,5],[3,2],[1,1]])

print(np.dot(x,y))
```

```
[[13 12]
 [18 17]]
```

  - Inner product of 2 arrays 1D:

```
v=np.array([2,3])
w=np.array([4,1])


print(v.dot(w))
print(np.dot(v, w))
```

```
11
11
```

    - dot() can be applied as **function** of NumPy module or as a **method** of an array object

Numpy – Numerical Operations

- **Elementwise** operations
  - Comparison

```
a=np.array([1,2,3,5])
b=np.array([3,4,2,5])
a==b
```

```
array([False, False, False,  True])
```

    - **np.array_equal()** has to be applied for an array-wise comparison

```
a=np.array([1,2,3,5])
b=np.array([3,4,2,5])
np.array_equal(a,b)
```

```
False
```

  - Logical Operations
    - and
    - or

```
a=np.array([1, 0, 0, 1], dtype=bool)
b=np.array([0, 1, 0, 1], dtype=bool)

print(np.logical_or(a,b))        #Logical OR

print(np.logical_and(a,b))       #Logical AND
```

```
[ True  True False  True]
[False False False  True]
```


Numpy – Numerical Operations

- **Elementwise** operations
  - Other (some examples):
    - sin()
    - log()
    - exp()
    - …

```
a=np.array([1,2,3,4,5])
print(np.sin(a))
print(np.log(a))
print(np.exp(a))
```

```
[ 0.84147098  0.90929743  0.14112001 -0.7568025  -0.95892427]
[0.         0.69314718 1.09861229 1.38629436 1.60943791]
[  2.71828183   7.3890561   20.08553692  54.59815003 148.4131591 ]
```

Numpy – Numerical Operations

- Other operations:
  - Transpose
    - Operator that flips a matrix over its diagonal
      - Switches the row and column indices of the matrix A by producing another matrix A^T
      - It is a view of the original array
      - Often applied/required

```
1 a=np.array([[1,2,3],[4,5,6]])
```

```
1 a
```

```
array([[1, 2, 3],
       [4, 5, 6]])
```

```
1 a.T
```

```
array([[1, 4],
       [2, 5],
       [3, 6]])
```


Numpy – Numerical Operations

- Sum
  - all the elements of an array

```
a=np.arange(5)
np.sum(a)
```

```
10
```

```
a=np.arange(5)
a.sum()
```

```
10
```

*sum() is other example that can be applied as **function** of NumPy module or as a **method** of an array object.*

https://numpy.org/doc/stable/reference/generated/numpy.sum.html#numpy.sum

  - sum by row

```
a=np.array([[2,3,4],[5,6,7]])
print(a)

np.sum(a,axis=1)
```

```
[[2 3 4]
 [5 6 7]]

array([ 9, 18])
```

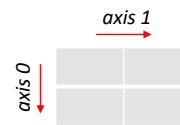
  - columns

```
a=np.array([[2,3,4],[5,6,7]])
print(a)

np.sum(a,axis=0)
```

```
[[2 3 4]
 [5 6 7]]

array([ 7,  9, 11])
```



If axis is not defined, sum() sums all the elements as a flattened array.

  - sum of a given row/column

```
a=np.array([[2,3,4],[5,6,7]])
print(a)

np.sum(a[:,2])
```

```
[[2 3 4]
 [5 6 7]]

11
```

Numpy – Numerical Operations

- min/max

```
1 a=np.array([[1,3,5],[6, 9, 11]])
2 print('Max: ', a.max())
3 print('Max: ', np.max(a))
```

```
Max:  11
Max:  11
```

```
1 print('Min: ', a.min())
2 print('Min: ', np.min(a))
```

```
Min:  1
Min:  1
```

- index of min/max

```
1 b=np.array([2,4,3,10,1,2,])
2 print('index of max: ',b.argmax())
3 print('index of min: ',b.argmin())
```

```
index of max:  3
index of min:  4
```




Exercise 12

Exercise 12

- Create 2 Numpy arrays based on the following lists:
  - [1,2,3]; [4,5,6]
    - multiply (element wise) the 2 arrays
    - multiply (array wise) the 2 arrays

Exercise 12

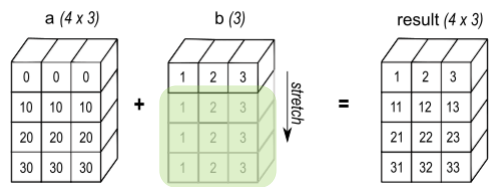
- Compare the 2 arrays
  - Element wise
  - array-wise



End - Exercise 12

Numpy – Numerical Operations

- Broadcasting
  - Basic operations are element wise (arrays with the same size)
  - Nevertheless it is also possible to do operations on arrays of different sizes since NumPy **can transform those arrays into the same size**
    - Broacasting (this automatic transformation)



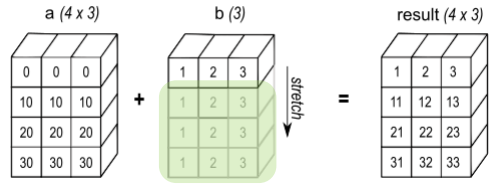
https://numpy.org/doc/stable/user/basics.broadcasting.html

```
1 x=np.array([[0,0,0],[1,2,3],[4,5,6]])
2 y=np.array([10,11,12])
3 print(x+y)
```

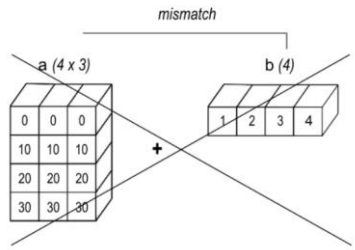
```
[[10 11 12]
 [11 13 15]
 [14 16 18]]
```

Numpy – Numerical Operations

- Broadcasting
  -



https://numpy.org/doc/stable/user/basics.broadcasting.html

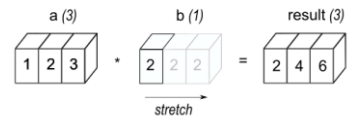


Numpy – Array Shape

- Broadcast
  - The simplest broadcasting
    - scalar / array

```
1 a=np.array([1,2,3,4])
2 b=a+2
3 print('add a scalar to a 1D array: \n', b)
```

```
add a scalar to a 1D array:
 [3 4 5 6]
```




```
1 a=np.array([[1,2,3],[4,5,6]])
2 b=a+3
3 print('add a scalar to a 2D array: \n',b)
```

```
add a scalar to a 2D array:
 [[4 5 6]
 [7 8 9]]
```

Array Shape

Numpy – Array Shape

- Sorting data
  - this is a very frequent operation
  - it is possible to sort data in an array:
    - by row/column

e.g.: first row

```
1 a=np.array([[4,3,5],[1,2,1]])
2 b=np.sort(a[0,:])
3 print(b)
```

```
[3 4 5]
```

e.g.: first column

```
1 b=np.sort(a[:,0])
2 print(b)
```

```
[1 4]
```

```
[[4 3 5]
 [1 2 1]]
```

    - all rows / columns

e.g.: by row

```
1 c=np.sort(a,axis=1)
2 print(c)
```

```
[[3 4 5]
 [1 1 2]]
```

e.g.: by column

```
1 c=np.sort(a,axis=0)
2 print(c)
```

```
[[1 2 1]
 [4 3 5]]
```

Numpy – Array Shape

- Flattening

```
a=np.array([[1,2,3],[4,5,6]])
print(a)
```

```
[[1 2 3]
 [4 5 6]]
```

  - transform a multidimensional array in an array 1D

```
print(a.ravel())
```

```
[1 2 3 4 5 6]
```

- Reshape
  - The inverse operation of flattening
  - very important/useful
  - dimensions are defined through a tuple

```
print(a.reshape(2,3))
```

```
[[1 2 3]
 [4 5 6]]
```

- Resizing
  - an array can be resized
    - the new elements are 0

```
a=np.arange(4)
a.resize(8,)
a
```

```
array([0, 1, 2, 3, 0, 0, 0, 0])
```




Exercise 13

Exercise 13

- Create the array:

```
[[2 5 7]
 [8 3 5]
 [9 1 1]]
```

  - sort each row of the array:

Exercise 13

- calculate the respective transpose:

- Define c=[0,1,2].
  - How to perform the addition to the previous array (transpose of the original array)?
    - Broadcast



End - Exercise 13

Numpy – Loading data

- Loading Data
  - Text Files
    - e.g. *.txt
      - np.loadtxt()
      - np.savetxt()

data1.txt - Notepad

File Edit Format View Help

```
# year  Europe  Asia    America Africa
1900    200e3   800e3   100e3   100e3
1950    300e3   1000e3  140e3   130e3
2000    350e3   1200e3  160e3   180e3
2020    400e3   1400e3  180e3   200e3
```

```
1 population=np.loadtxt('data1.txt')
2 print(population)
```

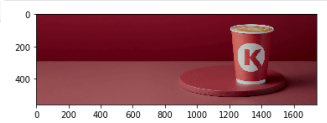
```
[[   1900.  200000.  800000.  100000.  100000.]
 [   1950.  300000. 1000000.  140000.  130000.]
 [   2000.  350000. 1200000.  160000.  180000.]
 [   2020.  400000. 1400000.  180000.  200000.]]
```

  - Image
    - plt.imread
    - plt.imsave

```
1 from matplotlib import pyplot as plt
2 img=plt.imread('coffee.png')
3 print(img.shape)
4 print(img.dtype)
```

```
(560, 1743, 4)
float32
```

```
1 plt.imshow(img);
```





Exercise 14

Exercise 14

- Compute some statistics of small dataset of patients
  - Create the array

Age	Weight
24	70
30	80
45	90
50	85
35	75

  - Dimensions of the dataset

Exercise 14

- Calculate the average age and weight

- Calculate the maximum and minimum weight

Exercise 14

- Find the patients older than 35

- Calculate the bmi

Age	Weight	Height
24	70	175
30	80	180
45	90	185
50	85	190
35	75	165



End - Exercise 14